

LocalBuka

Technical Case Study

1. Overview

LocalBuka is a food discovery platform that helps users find, compare, and decide where to eat. As an AI Engineer Intern here, you would work across multiple areas: building intelligent recommendation/search features, designing conversational assistants for users, and integrating AI components into our backend systems.

This case study is designed to give you room to show how you think and work, not just whether you can produce a “correct” answer. It has three parts. Please attempt all three, but we care more about the quality and clarity of your reasoning than about a perfect, production-ready solution.

2. What You Will Submit

- A link to a public (or shared) GitHub repository containing your code.
- A short written document (PDF, Word, or Markdown file in the repo) explaining your approach, decisions, and trade-offs — see Part 3 for what this should cover.
- Clear instructions in your repo's README on how to run your code, even if it does not run perfectly.

3. Part 1 — Recommendation / Search Logic (Core Task)

LocalBuka wants to recommend restaurants and dishes to users based on their preferences and behavior.

Your task:

1. Design and implement a simple recommendation function that takes a user's preferences (e.g. cuisine type, price range, past orders, dietary restrictions) and a list of available restaurants/dishes (you may invent realistic sample data — at least 20 sample restaurants/dishes), and returns a ranked list of recommendations.
2. You may use simple rule-based logic or an LLM-based approach (e.g. calling an AI model with a prompt) — choose whichever you can justify well. There is no single “correct” method.
3. Include at least 3 sample test cases showing different users getting different recommendations, with the output shown (in your README or a results file).

In your written explanation, address:

- Why you chose this approach over alternatives.
- How you would evaluate whether the recommendations are actually good, once live.
- How this would need to change if LocalBuka had 1 million users instead of 20 sample records.

4. Part 2 — Conversational Assistant Prototype

LocalBuka wants a simple chat assistant that helps users find food (e.g. “I want something spicy and cheap near me”).

Your task:

4. Build a basic working prototype of a chat assistant that can take a user's free-text message and respond helpfully, using the sample restaurant/dish data from Part 1 (or new sample data).
5. You may use any approach: a call to an LLM API (e.g. OpenAI, Anthropic, or any provider with a free tier), a simple rule/keyword-based bot, or a hybrid. If you use a paid API and don't want to spend money, it is acceptable to show the code and explain how it would work, with mocked/sample responses instead of live calls.
6. It does not need a fancy interface — a command-line chat.

In your written explanation, address:

- What could go wrong with this assistant in production (e.g. giving wrong info, inappropriate responses), and how you would guard against it.
- How you would keep API costs under control if using a paid LLM provider.

5. Part 3 — Written Reflection (Required)

In a single document (1–2 pages is enough), please answer:

7. Walk me through your overall approach and the key decisions you made for Parts 1 and 2.
8. What would you do differently, or build out further, with more time?
9. Describe one real risk or failure mode of using AI features in a consumer product like LocalBuka (e.g. bias, hallucination, data privacy, cost), and how you'd mitigate it.
10. Briefly describe a time (from past work, school, or personal projects) you had to debug or fix an AI/ML system that wasn't working as expected. What was the issue and how did you find it?

6. Evaluation Criteria

I will be assessing:

Area	What we're looking for
Problem-solving approach	How clearly you reason through the task, structure your solution, and justify your choices.
Code quality	Whether the code is organized and readable — not whether it's perfect or production-grade.
AI/ML judgment	Whether your approach to the recommendation and assistant tasks is sensible and appropriately scoped.

Communication	Whether your written explanation is clear enough for a non-technical reviewer to follow your thinking.
Awareness of real-world constraints	Whether you consider cost, scale, risk, and failure modes — not just “does it run.”